



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year:2025), B.Sc. in CSE (Day)

Lab Performance 14: Genetic Algorithm

CourseTitle: Artificial Intelligence
Lab Course Code: CSE-316 **Section:221-14**

NAME	ID
Abu Bakkar Siddik	221902265

Problem Statement: Given a target integer T and a fixed list length k, evolve a list of k numbers (0–9) whose product equals T.

Case# 1Input:

T = 12

k = 3

Case# 1

Output: 2 3 2

Case# 2Input:

T = 18

k = 3

Case# 2Output: 3 3 2

CODE:

```
import random
pop_size = 100
mutation_rate = 0.1
max_gen = 1000

def create_individual(k):
    return [random.randint(1, 9) for _ in range(k)]

def product(individual):
    result = 1
    for num in individual:
        result *= num
    return result

def fitness(individual, t):
    return abs(t - product(individual))

def mutate(individual):
    idx = random.randint(0, len(individual) - 1)
    individual[idx] = random.randint(1, 9)
    return individual

def crossover(p1, p2):
    split = random.randint(1, len(p1) - 1)
    return p1[:split] + p2[split:]

def genetic_algorithm(t, k):
    population = [create_individual(k) for _ in range(pop_size)]
```

```

for generation in range(max_gen):
    population.sort(key=lambda ind: fitness(ind, t))
    best = population[0]

    if product(best) == t:
        print(f"solution found in generation {generation}")
        return best

    next_gen = population[:10] # elitism: keep top 10
    while len(next_gen) < pop_size:
        parent1 = random.choice(population[:50])
        parent2 = random.choice(population[:50])
        child = crossover(parent1, parent2)
        if random.random() < mutation_rate:
            child = mutate(child)
        next_gen.append(child)
    population = next_gen

return None

def main():
    t = int(input("enter target product (t): "))
    k = int(input("enter number of elements (k): "))
    result = genetic_algorithm(t, k)
    if result:
        print("result:", result)
        print("product:", product(result))
    else:
        print("no solution found.")

if __name__ == "__main__":
    main()

```

OUTPUT:

```

enter target product (t): 12
enter number of elements (k): 3
solution found in generation 0
result: [3, 1, 4]
product: 12

=== Code Execution Successful ===

```